

# Modélisation et simulation distribuée du mouvement d'un banc de poissons

Yohann Front-Reignier, Iram Madani-Fouatih

**Résumé**—Ce rapport synthétise l'ensemble du travail de conception ainsi que d'implémentation d'une simulation distribuée du mouvement d'un banc de poisson. Nous avons réalisé une implémentation utilisant trois stratégies : une approche séquentielle, une parallélisation distribuée via MPI de la grille spatiale et une parallélisation à mémoire partagée utilisant Kokkos.

## I. INTRODUCTION

Le mouvement d'un banc de poissons ou d'un vol groupé d'oiseaux sont des phénomènes très importants à étudier. En effet, cette étude peut mener à la création de nouveaux algorithmes bio-inspirés pour les modéliser et les simuler afin de mieux comprendre certains de leurs fonctionnements et ainsi pouvoir les répliquer dans l'industrie, notamment l'aéronautique.

Le contrôle à distance de plusieurs drones ou d'essaims de drones sans contrôle humain est aujourd'hui assez répandu. De plus, il existe aussi des zones sans tour de contrôle, où les avions doivent être capables de s'auto-gérer. C'est pour répondre à ce genre de problèmes que nous nous intéressons à la simulation de systèmes existant dans le monde animal afin de pouvoir les répliquer et les utiliser.

L'étude de ces systèmes d'auto-organisation et de flocking a mené à la création du modèle des Boids par C. Reynolds en 1987, qui permet de simuler informatiquement ces comportements. Les boids suivent trois règles simples que nous détaillerons plus tard dans ce rapport.

En respectant ces trois règles, le boid calcule sa direction et sa vitesse. Chaque boid réalise ce calcul, ce qui crée des comportements dits émergents, c'est-à-dire que certains boids vont agir de sorte à créer des motifs sans avoir de contrôle central leur demandant de les produire. Il est donc difficile de prévoir le comportement d'un boid en le regardant individuellement. C'est ce qu'on appelle un système complexe.

Une grande partie des travaux actuels consiste donc à essayer d'améliorer cette simulation afin de pouvoir gérer un plus grand nombre de boids sans trop augmenter le temps de calcul. En effet, comme chaque boid doit prendre en compte ses voisins pour déterminer son comportement, le calcul peut vite devenir très coûteux lorsque la taille du banc augmente.

C'est pourquoi plusieurs approches ont été proposées pour accélérer ce genre de simulation. Certaines utilisent le calcul parallèle sur GPU, tandis que d'autres reposent sur une répartition du calcul sur plusieurs cœurs ou sur plusieurs machines. L'idée est alors de diviser l'espace de simulation en plusieurs parties afin que chaque unité de calcul puisse traiter une portion du banc de poissons.

Cependant, cette répartition du calcul pose plusieurs difficultés. Il faut notamment gérer les interactions entre boids situés dans des zones différentes, ainsi que les échanges d'informations lorsque les individus se déplacent d'un sous-domaine à un autre. Il faut aussi conserver un comportement réaliste, sans trop dégrader la qualité de la simulation.

Dans ce travail, nous allons donc nous intéresser à la modélisation d'un banc de poissons à l'aide du modèle des boids, puis à sa simulation distribuée. L'objectif est de comprendre comment ce modèle peut être adapté à une architecture parallèle afin d'améliorer les performances de calcul tout en gardant un comportement collectif cohérent.

Pour cela, le rapport sera organisé de la manière suivante : fondements théoriques, implémentations, comparative, défis, benchmarking, puis conclusion.

## II. FONDEMENTS THÉORIQUES

Nous formalisons le modèle mathématique des boids, sa complexité algorithmique et les défis liés au parallélisme.

Afin de représenter au mieux les différents comportements du banc de poissons, nous avons utilisé des formules mathématiques associées aux règles de flocking. Pour chaque boid, le calcul repose sur les interactions avec ses voisins proches, ce qui permet de déterminer sa nouvelle direction et sa nouvelle vitesse.

### 1) Règle 1 : Séparation

Le calcul de la séparation est nécessaire pour trouver notre vecteur de répulsion afin d'éviter les collisions. Nous avons utilisé la formule suivante :

$$\vec{s} = \frac{1}{N} \sum_{i=1}^N \frac{\vec{p} - \vec{p}_i}{|\vec{p} - \vec{p}_i|} \quad (1)$$

où :

- $\vec{p}$  : position du boid courant,
- $\vec{p}_i$  : position du voisin  $i$ ,
- $N$  : nombre de voisins dans le rayon de perception,
- la force de steering finale :  $\vec{F}_s = (\hat{s} \cdot v_{max} - \vec{v})$ , limitée à  $F_{max}$  la force maximale.

### 2) Règle 2 : Alignement

On calcule la vitesse moyenne des voisins afin de suivre leur direction et pouvoir ajuster la vitesse. Nous avons utilisé la formule suivante :

$$\vec{a} = \frac{1}{N} \sum_{i=1}^N \vec{v}_i \quad (2)$$

où :

- $\vec{v}_i$  : vélocité du voisin  $i$ ,
- $N$  : nombre de voisins,
- force de steering :  $\vec{F}a = (\hat{a} \cdot v_{max} - \vec{v})$ , limitée à  $F_{max}$  la force maximale.

### 3) Règle 3 : Cohésion

On calcule le centre de masse des voisins afin de faire en sorte de diriger l'agent vers ce centre de masse.

$$\vec{c} = \frac{1}{N} \sum_{i=1}^N \vec{p}_i \quad (3)$$

où :

- $\vec{c}$  : centre de masse des voisins,
- $\vec{p}_i$  : position du boid voisin  $i$ ,
- $N$  : nombre de voisins dans le rayon de perception,
- la force de steering finale :  $\vec{F}c = (\vec{c} - \vec{p} \cdot v_{max} - \vec{v})$ , limitée à  $F_{max}$  la force maximale.

Paramètres clés :

- perceptionRadius : rayon de détection des voisins,
- maxSpeed : vitesse maximale du boid,
- maxForce : force maximale applicable,
- separationWeight : poids de la règle de séparation,
- alignmentWeight : poids de la règle d'alignement,
- cohesionWeight : poids de la règle de cohésion.

Pour la simulation temporelle, nous utilisons une intégration d'Euler explicite afin de mettre à jour la position et la vitesse de chaque boid à chaque pas de temps :

$$\mathbf{x}_i(t + \Delta t) = \mathbf{x}_i(t) + \mathbf{v}_i(t) \cdot \Delta t \quad (4)$$

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \mathbf{a}_i(t) \cdot \Delta t \quad (5)$$

Du point de vue algorithmique, la version naïve de cette simulation a une complexité en  $O(N^2)$ , car chaque boid doit comparer sa position avec tous les autres boids. Afin de réduire ce coût, on peut utiliser une grille spatiale ou une autre structure de partition de l'espace, ce qui permet en moyenne d'obtenir une complexité plus proche de  $O(N)$  selon la densité de répartition des agents.

Enfin, la parallélisation de ce type de simulation pose plusieurs défis. Il faut trouver un bon compromis entre communication et calcul, gérer la synchronisation entre les différentes unités de calcul, et choisir une topologie de décomposition adaptée. En effet, si la partition de l'espace est mal choisie, les échanges entre sous-domaines peuvent devenir trop fréquents et ralentir fortement la simulation.

## III. ARCHITECTURE ET IMPLÉMENTATION

### A. Version séquentielle

La version séquentielle constitue notre baseline de référence : toute la simulation est exécutée dans un seul processus, sans communication réseau. Cette version permet de valider le modèle de Reynolds et de mesurer un temps de référence avant parallélisation distribuée.

L'architecture repose sur trois classes principales : `Vector2D` pour les opérations géométriques, `Boid` pour le

comportement individuel, et `Flock` pour la gestion collective. La classe `Flock` maintient la liste des boids ainsi qu'une grille spatiale (`SpatialGrid`) utilisée pour accélérer la recherche de voisins.

À chaque pas de temps, l'algorithme suit trois étapes : (1) reconstruction de la grille spatiale, (2) calcul des forces de séparation, alignement et cohésion, puis (3) intégration des positions/vitesses avec un schéma d'Euler explicite. Le wrapping toroïdal est ensuite appliqué aux frontières pour conserver les boids dans le domaine de simulation.

Cette organisation permet de réduire fortement le coût du voisinage : au lieu d'une recherche naïve en  $O(N^2)$ , la grille limite les comparaisons aux cellules proches, ce qui donne en pratique un comportement proche de  $O(N \cdot k)$ , où  $k$  est le nombre moyen de voisins pertinents.

### B. Version Parallèle

La version parallèle combine un parallélisme distribué (MPI) entre processus et un parallélisme local (Kokkos/OpenMP) à l'intérieur de chaque processus. L'objectif est de conserver la cohérence physique du modèle tout en améliorant les performances quand le nombre de boids augmente.

L'espace 2D est découpé en sous-domaines, chacun géré par un rang MPI. Chaque rang ne met à jour que ses boids locaux, mais doit tenir compte des boids proches des frontières. Pour cela, un échange de halos est effectué entre rangs voisins via `MPI_Sendrecv`. Les boids qui traversent une frontière de sous-domaine sont ensuite transférés au bon rang par migration (`MPI_Alltoallv`).

Sur chaque rang, le calcul des forces et l'intégration des positions sont parallélisés avec `Kokkos::parallel_for`. Ce choix permet d'exploiter la mémoire partagée (backend OpenMP) sans modifier la logique métier des boids.

Une itération complète suit la séquence suivante : échange de halos, reconstruction de la grille locale (boids locaux + halo), calcul des forces, intégration des positions, migration inter-domaines, puis synchronisation du nombre global de boids (`MPI_Allreduce`). Cette chaîne garantit que chaque rang dispose des informations nécessaires pour calculer des interactions locales cohérentes.

## IV. ANALYSE COMPARATIVE

Cette partie compare les performances des trois approches à partir des métriques de temps, speedup et efficacité.

### V. DÉFIS RENCONTRÉS ET RÉOLUTIONS

Nous détaillons les principaux problèmes techniques rencontrés, les corrections appliquées et les points restant ouverts.

### VI. BENCHMARKING COMPLET

Nous présentons la méthodologie expérimentale, les résultats statistiques et leur interprétation via les lois de scalabilité.

## VII. DISCUSSION

Cette section analyse les résultats, explicite les limites du projet et propose des pistes d'amélioration.

## VIII. CONCLUSION

Nous synthétisons les contributions du projet et répondons aux questions centrales sur l’efficacité des approches parallèles.

## IX. RÉFÉRENCES BIBLIOGRAPHIQUES

Cette section liste les sources scientifiques et techniques ayant servi de base au projet.

### RÉFÉRENCES

- [1] C. W. Reynolds, “Flocks, herds and schools : A distributed behavioral model,” in *Proc. SIGGRAPH*, 1987.
- [2] C. W. Reynolds, “Steering behaviors for autonomous characters,” in *Proc. Game Developers Conf.*, 1999.
- [3] D. S. Hollman *et al.*, “Kokkos : Enabling manycore performance portability through polymorphic memory access patterns,” *J. Parallel Distrib. Comput.*, vol. 74, no. 12, pp. 3202–3216, 2014.
- [4] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI : Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 1998.
- [5] OpenMP Architecture Review Board, “OpenMP Application Program Interface,” Version 5.2, 2023. [Online]. Available : <https://www.openmp.org/>
- [6] L. Gomila, “SFML - Simple and Fast Multimedia Library,” 2024. [Online]. Available : <https://www.sfml-dev.org>
- [7] Google, “GoogleTest User’s Guide,” 2024. [Online]. Available : <https://google.github.io/googletest/>
- [8] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.
- [9] M. Buckland, *Programming Game AI by Example*. Course Technology, 2004.
- [10] D. Shiffman, *The Nature of Code : Simulating Natural Systems with Processing*. 2012. [Online]. Available : <https://natureofcode.com/>
- [11] Kitware, “CMake Build System,” 2024. [Online]. Available : <https://cmake.org/>